

Experiment No. 5

Aim of the Experiment: Develop an RMI application which accepts a string or a number and checks that string or number is palindrome or not.

Objective:

To Develop program using RMI application

Course Outcome Addressed: CO5: Invoke the remote methods in an application using Remote Method Invocation (RMI)

Theory:

Palindrome number:

A **palindrome number** is a number that is same after reverse. For example 545, 151, 34543, 343, 171, 48984 are the palindrome numbers. It can also be a string like LOL, MADAM etc.

Remote method invocation(RMI):

Remote method invocation(RMI) allow a java object to invoke method on an object running on another machine. RMI provide remote communication between java program. RMI is used for building distributed application.

Concept of RMI application:

A RMI application can be divided into two part, Client program and Server program. A Server program creates some remote object, make their references available for the client to invoke method on it. A Client program make request for remote objects on server and invoke method on them. Stub and Skeleton are two important object used for communication with remote object.

Stub:

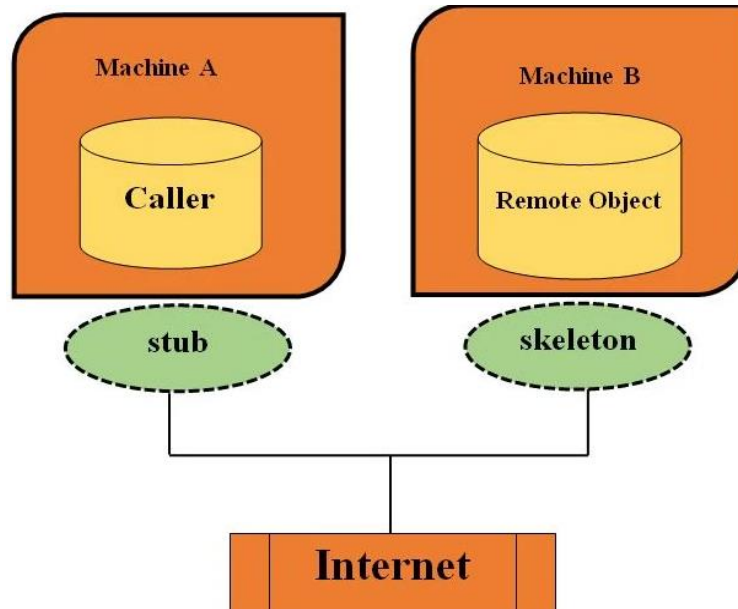
In RMI, a stub is an object that is used as a Gateway for the client-side. All the outgoing request are sent through it. When a client invokes the method on the stub object following things are performed internally:

1. A connection is established using Remote Virtual Machine.
2. It then transmits the parameters to the Remote Virtual Machine. This is also known as Marshals
3. After the 2nd step, it then waits for the output.
4. Now it reads the value or exception which is come as an output.
5. At last, it returns the value to the client.

Skeleton

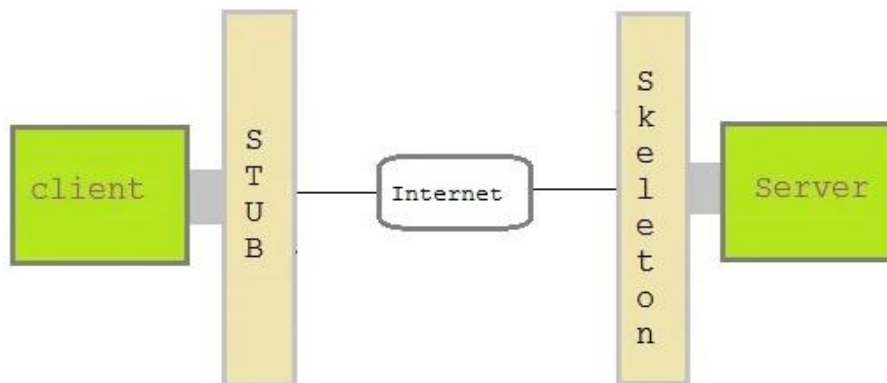
In RMI, a skeleton is an object that is used as a Gateway for the server-side. All the incoming request are sent through it. When a Server invokes the method on the skeleton object following things are performed internally:

1. All the Parameters are read for the remote method.
2. The method is invoked on the remote object.
3. It then writes and transmits the parameters for the result. This is also known as Marshals.



Stub and Skeleton

Stub act as a gateway for Client program. It resides on Client side and communicate with Skeleton object. It establish the connection between remote object and transmit request to it. Skeleton object resides on server program. It is responsible for passing request from Stub to remote object.



Creating a Simple RMI application involves following steps:-

- Define a remote interface.
- Implementing remote interface.
- create and start remote application
- create and start client application

Algorithm:

Department of Electronics & Telecommunication Engineering

Subject: Advanced Java Programming Lab (Elective - II) (304198)(C)

Exp. No.5

SERVER SIDE:

Step 1: Start
Step 2: Define the class rmiserver
Step 3: Create the object twox in try
Step 4: Register the object twox
Step 5: Display the exception in catch
Step 6: Stop

CLIENT SIDE:

Step 1: Start
Step 2: Define the class rmiclient
Step 3: Initialize the string s1 in try
Step 4: Create and Initialize the object onex
Step 5: Assign the value to m by calling the method palin
Step 6: Check whether the string is palindrome or not
Step 7: Display whether the string is palindrome or not
Step 8: Display the exception in catch
Step 9: Stop

SOURCE CODE:**one.java**

```
import java.rmi.*;
interface one extends Remote
{
    public int palin(String a) throws RemoteException;
}
```

two.java

```
import java.rmi.*;
import java.lang.*;
import java.rmi.server.*;
public class two extends UnicastRemoteObject implements one
{
    public two() throws RemoteException { }
    public int palin(String a) throws RemoteException
    {
        System.out.println("Hello");
        StringBuffer str = new StringBuffer(a);
        String str1 = str.toString();
        System.out.println("Print : " + str1.toString());
    }
}
```

Department of Electronics & Telecommunication Engineering

Subject: Advanced Java Programming Lab (Elective - II) (304198)(C)

Exp. No.5

```
StringBuffer str2 = str.reverse();
System.out.println("Print : " + str2.toString());
int b = str1.compareTo(str2.toString());
System.out.println("Print : " + b);
if (b == 0)
    return 1;
else
    return 0;
}
}
```

rmiserver.java

```
import java.io.*;
import java.rmi.*;
import java.net.*;
public class rmiserver
{
    public static void main(String args[]) throws Exception
    {
        try
        {
            two twox = new two();
            Naming.bind("palin", twox);
            System.out.println("Object registered");
        }
        catch(Exception e)
        {
            System.out.println("Exception" + e);
        }
    }
}
```

rmiclient.java

```
import java.io.*;
import java.rmi.*;
import java.net.*;
public class rmiclient
{
    public static void main(String args[]) throws Exception
    {
        try
        {
```

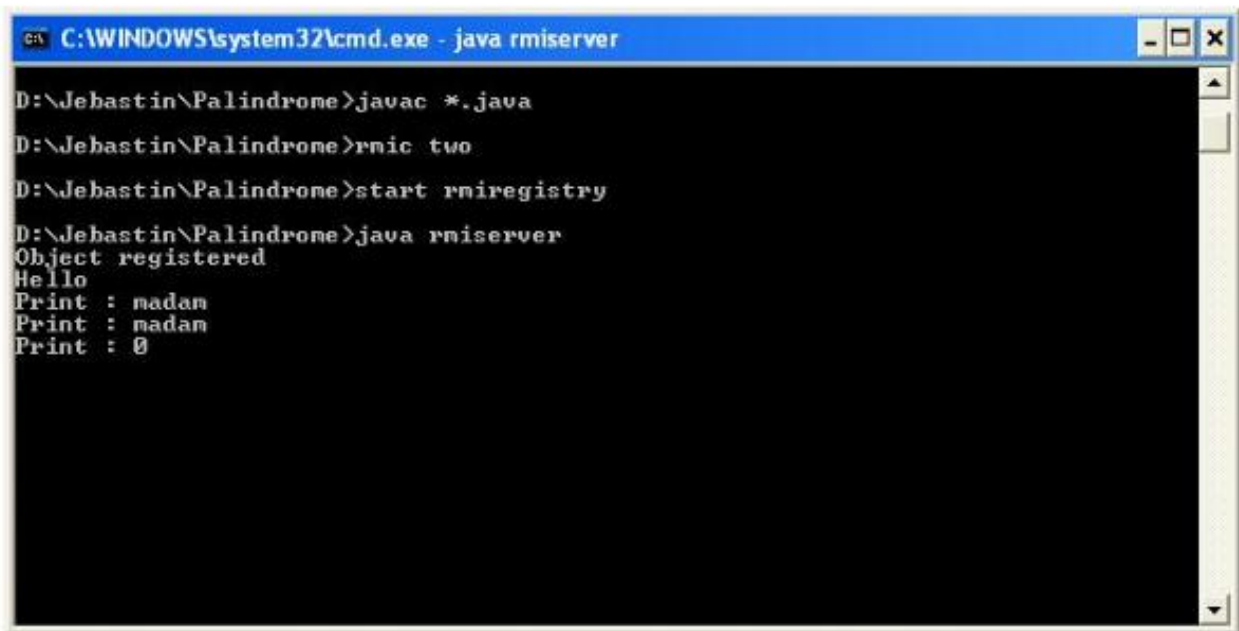
Department of Electronics & Telecommunication Engineering

Subject: Advanced Java Programming Lab (Elective - II) (304198)(C)

Exp. No.5

```
String s1 = "rmi://localhost/palin";
one onex = (one)Naming.lookup(s1);
int m = onex.palin("madam");
System.out.println("Print : " + m);
if (m == 1)
{
    System.out.println("The given string is a Palindrome");
}
else
{
    System.out.println("The given string is not a Palindrome");
}
}
catch (Exception e)
{
    System.out.println("Exception" + e);
}
}
```

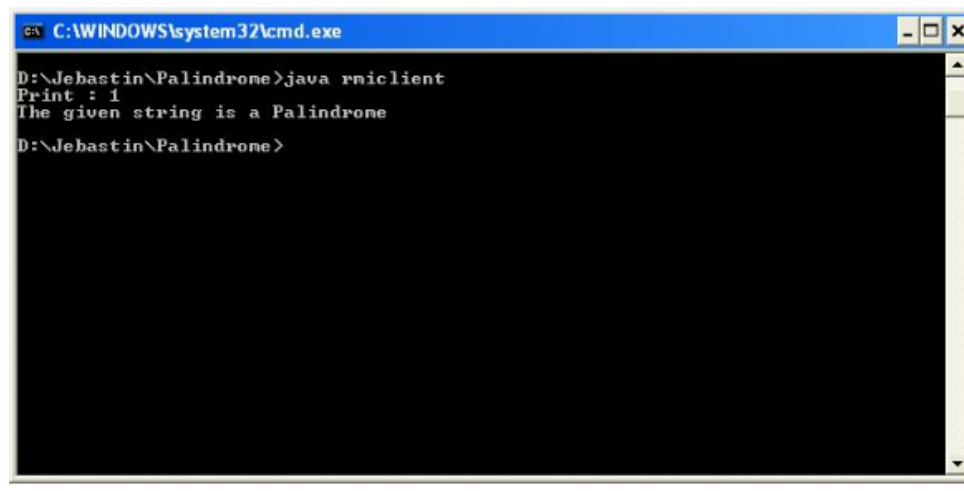
OUTPUT:**SERVER SIDE:**



```
C:\WINDOWS\system32\cmd.exe - java rmiserver

D:\Jebastin\Palindrome>javac *.java
D:\Jebastin\Palindrome>rmic two
D:\Jebastin\Palindrome>start rniregistry
D:\Jebastin\Palindrome>java rmiserver
Object registered
Hello
Print : madam
Print : madam
Print : 0
```

CLIENT SIDE:



```
C:\WINDOWS\system32\cmd.exe

D:\Jebastin\Palindrome>java rmiclient
Print : 1
The given string is a Palindrome
D:\Jebastin\Palindrome>
```

PalindromeClass

```
package rmi;
import java.rmi.*;
import java.rmi.server.*;
public class PalindromeClass extends UnicastRemoteObject implements Remote,
PalindromeInterface{
    PalindromeClass() throws RemoteException{
        super();
    }
    public String palindromeCheck(String s) {
        String rev = "";
        for (int i = s.length()-1; i>=0; i--) {
            rev += s.charAt(i);
        }
        if (s.equals(rev)) {
            String k = "The word " + s + " is a palindrome !";
            return k;
        }
        else {
            String k = "The word " + s + " is not a palindrome !";
            return k;
        }
    }
}
```

PalindromeInterface

```
package rmi;
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface PalindromeInterface extends Remote{
    public String palindromeCheck(String s) throws RemoteException;
}
```

PalindromeServer

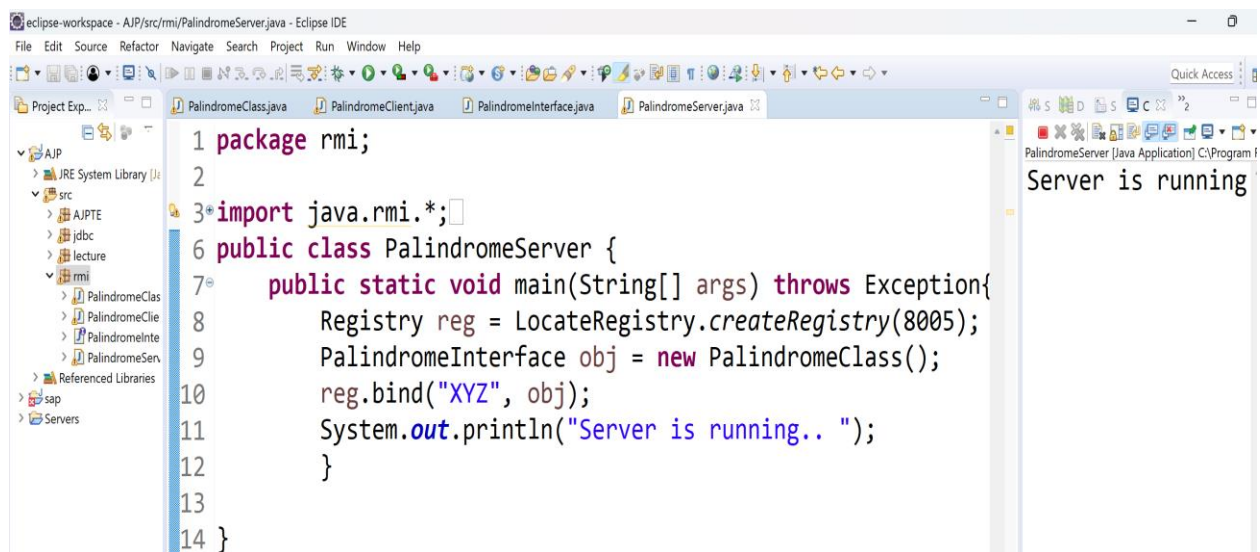
```
package rmi;
import java.rmi.*;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
public class PalindromeServer {
    public static void main(String[] args) throws Exception{
        Registry reg = LocateRegistry.createRegistry(5000);
        PalindromeInterface obj = new PalindromeClass();
        reg.bind("XYZ", obj);
        System.out.println("Server is running.. ");
    }
}
```

PalindromeClient

```
package rmi;
import java.rmi.*;
import java.rmi.registry.*;
public class PalindromeClient {
    public static void main(String[] args) throws Exception{
        PalindromeInterface obj1 = (PalindromeInterface)Naming.lookup("rmi://localhost:5000/XYZ");
        String ans1 = obj1.palindromeCheck("HELLO");
        String ans2 = obj1.palindromeCheck("RACECAR");
        String ans3 = obj1.palindromeCheck("MADAM");
        String ans4 = obj1.palindromeCheck("NAYAN");
        String ans5 = obj1.palindromeCheck("ANCHAL");
        System.out.println(ans1);
        System.out.println(ans2);
        System.out.println(ans3);
        System.out.println(ans4);
        System.out.println(ans5);
    }
}
```

OUTPUT:

SERVER SIDE:



The screenshot shows the Eclipse IDE with the following details:

- Project Explorer:** Shows a project named 'AJP' with a source folder 'src'. Inside 'src', there is a package 'rmi' containing files: 'PalindromeClass.java', 'PalindromeClient.java', 'PalindromeInterface.java', and 'PalindromeServer.java'.
- Editor:** Displays the code for 'PalindromeServer.java'. The code is as follows:

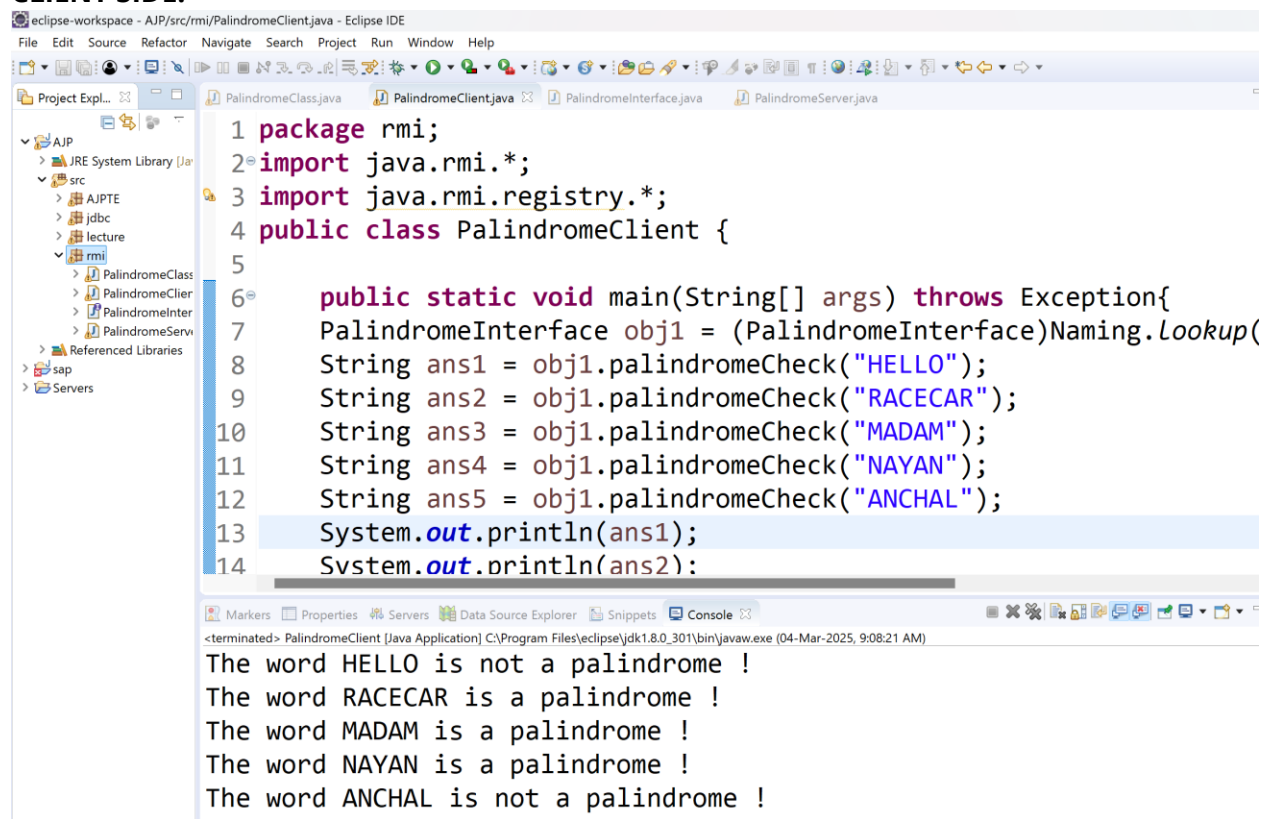

```
1 package rmi;
2
3 import java.rmi.*;
4
5
6 public class PalindromeServer {
7     public static void main(String[] args) throws Exception{
8         Registry reg = LocateRegistry.createRegistry(8005);
9         PalindromeInterface obj = new PalindromeClass();
10        reg.bind("XYZ", obj);
11        System.out.println("Server is running.. ");
12    }
13
14 }
```
- Console:** Shows the output of the server: "Server is running".

Department of Electronics & Telecommunication Engineering

Subject: Advanced Java Programming Lab (Elective - II) (304198)(C)

Exp. No.5

CLIENT SIDE:



```

1 package rmi;
2 import java.rmi.*;
3 import java.rmi.registry.*;
4 public class PalindromeClient {
5
6     public static void main(String[] args) throws Exception{
7         PalindromeInterface obj1 = (PalindromeInterface)Naming.lookup(
8             String ans1 = obj1.palindromeCheck("HELLO");
9             String ans2 = obj1.palindromeCheck("RACECAR");
10            String ans3 = obj1.palindromeCheck("MADAM");
11            String ans4 = obj1.palindromeCheck("NAYAN");
12            String ans5 = obj1.palindromeCheck("ANCHAL");
13            System.out.println(ans1);
14            System.out.println(ans2);

```

Console Output:

```

<terminated> PalindromeClient [Java Application] C:\Program Files\eclipse\jdk1.8.0_301\bin\javaw.exe (04-Mar-2025, 9:08:21 AM)
The word HELLO is not a palindrome !
The word RACECAR is a palindrome !
The word MADAM is a palindrome !
The word NAYAN is a palindrome !
The word ANCHAL is not a palindrome !

```

Conclusion:

References:

Herbert Schildt , “Java : The Complete Reference” Tata McGraw-Hill (7th Edition).

Questions:

1. What is Java Remote Method Invocation (RMI)?
2. What is RMI remote object?
3. How does RMI communicate with the remote object?
4. Explain Different layers of RMI architecture.
5. What Are The Different Types Of Classes That Are Used In RMI?